

Optimization of Logistic Operations in Decentralized Manual Warehouses

Abdelrahman Mohamed Aly Sobh* 30061A

Summary of Master's Thesis
A.Y. 2025/2026

Abstract

The efficient allocation of resources to tasks under operational constraints is a fundamental challenge that pervades a wide range of real-world domains, from **logistics** and manufacturing to healthcare and emergency response. In many such domains, the assignment problem is further complicated by spatial dependencies in which resources must physically travel between locations, and the time cost of these movements directly influences the feasibility, efficiency, productivity and the overall quality of any proposed schedule. When formalized rigorously, these problems belong to the class of **combinatorial optimization problems** (COPs), many of which are NP-hard and therefore intractable by exhaustive search even for moderate instance sizes.

Combinatorial optimization deals with problems with an *objective function* that has to be minimized or maximized over a set of combinatorial alternatives. The most naive way of obtaining the optimal solution is to list all the feasible solutions, evaluating them using the objective function and selecting the optimal one. Nevertheless, a brute-force approach lacks practicality when the size of the problem is too large, as the time needed to solve it grows exponentially with the problem size [And25].

Classical optimization methods, most notably **mixed-integer programming** (MIP) [Wol21], have long served as the gold standard for solving such problems to provable *optimality*. However, their computational cost grows prohibitively with the size and complexity of the instance; since they use the **branch and bound** (B&B) algorithm, rendering them impractical in time-critical or large-scale deployment scenarios.

Recently, the intersection of machine learning and combinatorial optimization has opened a promising alternative approach called "**neural combinatorial optimization** (NCO)" [And25, AAS25]. Rather than solving each problem instance from

*<https://github.com/Abudo-S/ManualWarehouseOptimization>.

scratch, learned solvers attempt to extract generalizable patterns from historical instances and exploit this knowledge to guide or replace the search process at inference time. Among the architectures that have proven a particular effectiveness for this purpose are **graph neural networks** (GNNs), which operate natively on graph-structured data and are naturally suited to encode the relational structure of scheduling and routing problems.

Neural combinatorial optimization attempts to learn good heuristics for solving a set of problems using neural network models. However, the incorporation of such algorithms in the conventional optimization framework has raised many questions related to their (1) performance (2) training data transferability (3) computational cost (4) generalization to large size instances (5) the experimental comparison with other methods such as exact algorithms, heuristics and metaheuristics.

Our thesis investigates the application of GNNs to a real-world operator-mission assignment problem characterized by spatial travel times, processing constraints, and planning horizon limits. We propose a neural solver based on graph attention networks v2 (GATv2) and connected multi-layer perceptron (MLP) heads, trained to approximate high-quality predictions. The training can be integrated with a separate post-processing procedure (decoder) that picks and checks the produced predictions in order to build the final solution; otherwise, an end-to-end (E2E) training can solve the decoding problem by fusing the decoding step into the model, so the final solution is automatically produced from initial raw data.

At inference time, solutions are constructed through a greedy scheduling procedure that explicitly accounts for operation’s required time, operator availability, and real-world physical constraints.

Our thesis was organized as follows:

- Part 1 (Modeling): formalizes the scheduling problem as a graph model and derives its algebraic mathematical programming formulation.
- Part 2 (Algorithms & Metrics): describes the exact MIP model, the GNN architectures, the inference methodology, the lower bound computation strategy, and the KPI definitions used throughout the evaluation.
- Part 3 (Experiments): presents the experimental results on real-world datasets, beginning with mini-batch experiments and scaling up to the full problem, with quantitative metrics and comparative qualitative analyses.

1. Index

- Problem Definition
- Thesis Objective
- Proposed Neural Scheduler Models
- Validation Metrics and Key Performance Indicators
- Description of Experiments
- Results
- Computational Environment and Used Technologies
- References

2. Problem Definition

In **manual warehouses**, a scale-variant batch of logistic operations (picking, unpicking) need to be geographically distributed on a fleet of a limited number of forklift operators without exceeding their shift. The primary goal is to find an optimal schedule that minimizes the overall *makespan* of these operations, minimizing also *the total number of operators* required to fulfill these operations (called "missions" or "orders"). Furthermore, the workload distribution need also to be balanced for all activated operators. Once the batch is processed and the optimized schedule is built, each operator is informed of his missions by the start of the next work day.

The problem extends classical bipartite matching by introducing spatiotemporal routing dynamics, meaning the feasibility and cost of an assignment depend heavily on the sequential physical movement of the operators.

Makespan (C_{max}): The makespan, defined as $\max(C_1, \dots, C_n)$, is equivalent to the completion time of the last job to leave the system. A minimum makespan usually implies a good utilization of the machine(s). [Pin08]

Where C_j is the completion time of mission j and n is the total number of missions that need to be scheduled.

2.1. System Entities

The operational environment consists of three primary entities: a **central depot**, a set of **missions**, and a pool of available **operators**. The central depot, conceptually designated as node 0, serves as the universal starting location for all available resources at the beginning of the scheduling period. Missions represent localized tasks that require a specific duration of active processing time to be completed. Operators are the mobile agents responsible for executing these missions, moving through the environment and processing tasks sequentially until the operational time horizon is reached.

2.2. Operational Constraints and Dynamics

The temporal progression of the system is governed by a combination of **travel times** and **processing times**. When an operator is assigned their first mission, they must incur an initial travel time penalty representing the distance from the base depot to the task location. For all subsequent assignments, the travel time is calculated based on the distance between the operator's last completed mission and their newly assigned target. Furthermore, all assignments are strictly bound by a planning **horizon constraint**, meaning an operator cannot be assigned a mission if the combined travel and processing time exceeds the remaining available time in the current shift.

2.3. Graph Conceptualization

The spatially constrained operator-mission assignment problem can be formally represented as an optimization problem on a **heterogeneous** directed multigraph [fig.2.1], denoted as $G=(V, E)$, where V is its vertex set and E represents its edges. The heterogeneity of the graph arises from the presence of two functionally distinct sets of vertices (nodes) and two distinct classes of edges, representing different physical and temporal properties of the system. [ZLZ24]

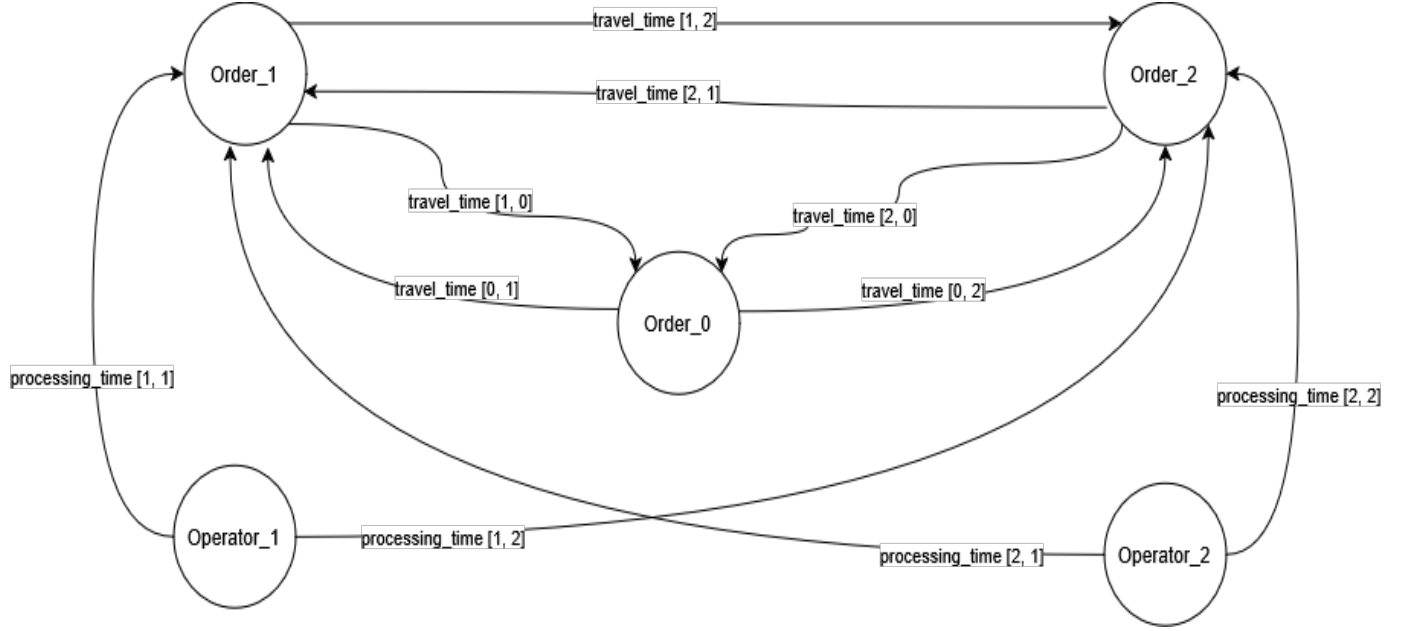


Figure 2.1: System Graph Representation.

The vertex set V is partitioned into two disjoint subsets, $V = I_{max} \cup J'$, representing the physical entities in the system:

Operator Nodes (I_{max}): The set of mobile agents available to be activated to execute tasks (e.g., $Operator_1, Operator_2, \dots$). An individual operator is denoted as $i \in I_{max}$.

Order Nodes (J'): The set of geographically distributed tasks (e.g., $Order_1, Order_2, \dots$) that require processing with spatial features. This set explicitly includes a specialized *depot node* (represented as $Order_0$), which serves as the physical origin point for all operators at the beginning of the scheduling horizon. An individual assignable order is denoted as $j \in J$, where $J = J' - \{Order_0\}$.

2.4. Edge Classes and Temporal Dynamics

The edge set E is similarly partitioned into two functional categories, $E = E_T \cup E_P$, which encode the spatial and processing constraints of the scheduling environment:

Travel Time Edges (E_T): These directed edges exclusively connect the nodes within the order subset J , such that $E_T \subseteq J \times J$. An edge $(j, j') \in E_T$ represents the spatial routing cost—specifically the time required to travel from the location of order j to order j' . Crucially, the graph includes travel edges originating from the depot node Order_0 to the other orders (e.g., $\text{travel_time}[0, 1]$), representing the initial dispatch time penalty incurred when an operator i is assigned their first task. Meanwhile, travel edges directed to the depot node Order_0 from other orders (e.g., $\text{travel_time}[1, 0]$) indicates the return-to-base constraint for an operator i .

Processing Time Edges (E_P): These directed edges connect operator nodes to order nodes, such that $E_P \subseteq I_{\max} \times J$. An edge $(i, j) \in E_P$ encodes the specific duration required for operator i to complete order j (e.g., $\text{processing_time}[1, 1]$).

While travel times in the current graph are assumed to be symmetric ($E_{Tj,j'} = E_{Tj',j}$), the problem is modeled on a directed graph (digraph). This explicit representation via dual arcs allows for the future integration of asymmetric routing constraints without any loss of generality.

2.5. System Objective

The optimization objective is to determine an optimal schedule—represented as a sequence of traversed edges—that maps the operators in $I_{\max}$ to the orders in J . A valid schedule must ensure that for any given operator $i \in I_{\max}$, the total time consumed does not exceed the maximum allowable operational horizon limit, denoted as H_{fixed} .

Specifically, the sum of the traversed travel time edges (E_T) and processing time edges (E_P) for operator i must satisfy:

$$\sum_{(j,j') \in E_{T,i}} \text{travel_time}(j, j') + \sum_{(i,j) \in E_{P,i}} \text{processing_time}(i, j) \leq H_{\text{fixed}}$$

3. Thesis Objective and Proposed Models

The main objective of this research is to address the complex scheduling and routing challenges inherent in modern warehouse logistics. From a business perspective, the overarching **goal** is to construct a robust operational schedule that simultaneously **minimize** the global completion time (*makespan*) and the required workforce (*operator activation*).

This problem is formally cast as a multi-criteria **optimization** task that seeks to strictly balance time efficiency against operational labor costs. The target minimizes a weighted sum of the global makespan Z and the total number of activated operators $\sum_{i \in I_{max}} y_i$. Hence, the *multi-criteria objective function* can be formalized as:

$$\text{Minimize } \alpha \cdot Z + \beta \cdot \sum_{i \in I_{max}} y_i \quad (3.1)$$

where α and β are weighting factors defining the explicit trade-off cost between minimizing execution time and minimizing staff utilization.

In the present business case, the primary objective is to minimize operator activation, with makespan minimization serving as a secondary goal. To enforce this hierarchy, the weight β must be sufficiently large relative to α . A naive approach to ensure that the solver prioritizes activation in all scenarios is to set $\beta = M$ and $\alpha = 1$, where M is a sufficiently large positive number (e.g., $1e5$). However, to maintain numerical stability and avoid excessively large coefficients, we only require β to exceed the maximum possible value of the makespan. Given that the makespan is strictly bounded by the shift duration limit H_{fixed} , we can effectively set $\beta = H_{\text{fixed}} + 1$ while maintaining $\alpha = 1$.

By setting $\beta = H_{\text{fixed}} + 1$, we guarantee that:

$$\beta \cdot y_i > \alpha \cdot \text{Makespan}$$

even if the makespan is at its absolute maximum. This effectively tells the solver: "saving one operator is always more valuable than saving any amount of time within a single shift."

3.1. The Trade-off between Makespan and Operator Activation

The **objective function** is parameterized by α and β , which represent the relative weights assigned to competing goals: a higher α prioritizes the minimization of latency (execution time), whereas a higher β prioritizes resource conservation (staff utilization).

With more active operators: Missions are split across many routes, each route is shorter, so the overall makespan decreases. But $\sum y_i$ increases (more operators used).

With fewer active operators: Missions are concentrated on few routes, and these routes get close to the shift capacity H_{fixed} , so makespan increases. But $\sum y_i$ decreases (lower labor / resource cost).

Because of this obvious trade-off and inter-coupling in multi-criteria optimization, the produced optimal schedule is defined *Pareto-optimal* schedule.

Pareto-optimal schedule: A schedule is called Pareto-optimal if it is not possible to decrease the value of one objective without increasing the value of the other. [[Pin08](#)]

4. Proposed Neural Scheduler Models

Traditionally, this optimization is approached using exact MIP models. While the exact MIP framework guarantees strict mathematical optimality for small-scale mini-batches (ranging from 20 to 50 routing examples), it suffers from severe combinatorial explosion. Consequently, the exact solver struggles to scale, becoming computationally intractable for realistic, full-batch sizes of ~ 1000 examples which are typical of industrial environments.

To overcome this fundamental scalability bottleneck, this thesis proposes the development of a fast, scalable neural combinatorial optimization solver. The core methodology relies on **imitation learning**, specifically "**offline behavioural cloning**": by training a neural model on the optimal schedules generated by the MIP solver for mini-batches, the network learns to imitate or approximate the exact optimality. The ultimate objective is to generalize this learned policy to produce valid, near-optimal schedules for real-world large-scale instances that are inaccessible to the exact solver. To thoroughly investigate this approach, we propose, implement, and rigorously compare two distinct GNN architectures:

- Static *one-shot* model.
- Sequential *recurrent* model.

4.1. One-Shot GNN Model

The **one-shot GNN model** is designed to predict the entire warehouse schedule in a single forward pass without sequential state updates. Given a heterogeneous graph representing the warehouse orders, available operators, and the physical parameters governing travel and processing times, the model outputs global probabilities for operator activation, task assignment, and task sequencing simultaneously.

Architecture Overview

As illustrated in the model architecture (fig. 4.1), the pipeline begins with a heterogeneous graph set containing distinct "order" and "operator" nodes. This graph is passed through a message-passing backbone consisting of three layers of graph attention networks v2 (GATv2). The GATv2 layers generate enriched node embeddings that capture the spatial distribution of orders and the mechanical capacities of the operators.

These embeddings are then fed into three independent multi-layer perceptron (MLP) prediction heads:

- **Activation MLP head:** Takes the operator embeddings and a heuristic boost (estimating minimum required operators) to output a sigmoid probability $\hat{P}_{active}$ indicating whether an operator should be deployed for the shift.

- **Assignment MLP head:** Concatenates operator and order embeddings to output a sigmoid probability matrix $\hat{P}_{assign}$, representing the likelihood of assigning order j to operator i .
- **Sequence MLP head:** Utilizes order embeddings and shared active operator scores to output a sigmoid probability matrix $\hat{P}_{seq}$, representing the likelihood of executing order j immediately after order j' .

Schedule Construction via Greedy Decoding

Because the one-shot model does not update its state sequentially, the raw probability matrices must be parsed into a physically valid, non-overlapping schedule. This is achieved using a **greedy decoding heuristic**. The decoding process transforms the continuous probabilities into discrete scheduling decisions while strictly enforcing temporal constraints (H_{fixed}) and spatial routing logic.

The greedy decoding algorithm operates as follows:

1. **Operator activation:** The model first evaluates the activation head output. Having the theoretical minimum number of operators (K), operators with an activation probability $\hat{P}_{active}$ exceeding a predefined threshold are deployed into the active workforce pool.
2. **Global assignment:** With the active fleet established, the algorithm groups the assignment probabilities ($\hat{P}_{assign}$) into quantized bins (rounded to 3 decimals) to handle tied predictions fairly. It iterates through all unassigned orders, greedily assigning each order to the active operator with the highest assignment probability. To prevent severe imbalances, the algorithm utilizes a cascading tie-breaker: if probabilities are equal, it prefers the operator with the highest current load to promote dense, localized spatial clusters.
3. **Local sequence routing:** Once the orders are clustered to specific operators, the decoder relies on the sequence probabilities ($\hat{P}_{seq}$) to establish the temporal routes. For a given operator, the decoder starts at the base and recursively selects the unvisited order with the highest sequence probability from the current node. Before finalizing a step, the physical constraints are rigidly checked: the accumulated travel time and processing time must not exceed h_{fixed} . If a sequence edge violates the shift horizon, it is discarded, and the decoder falls back to the next highest probable edge.
4. **Fallback protocol:** If the model fails to predict enough active operators, or if the active operators exhaust their temporal capacity before all orders are assigned, the greedy heuristic intervenes. Rejected orders are returned to a global pool, and if they can fit within the current activated staff, a refined *tail-insertion phase* is applied. Otherwise, the algorithm iteratively activates standby operators from the inactive pool, recalculating assignments until the entire batch is successfully scheduled.

This one-shot approach benefits from extremely fast parallel **inference**, as the entire probability landscape is generated simultaneously. However, because the prediction heads do not

dynamically "see" the intermediate choices made by the greedy decoder, the model heavily relies on the post-processing rules to resolve spatial conflicts and capacity violations.

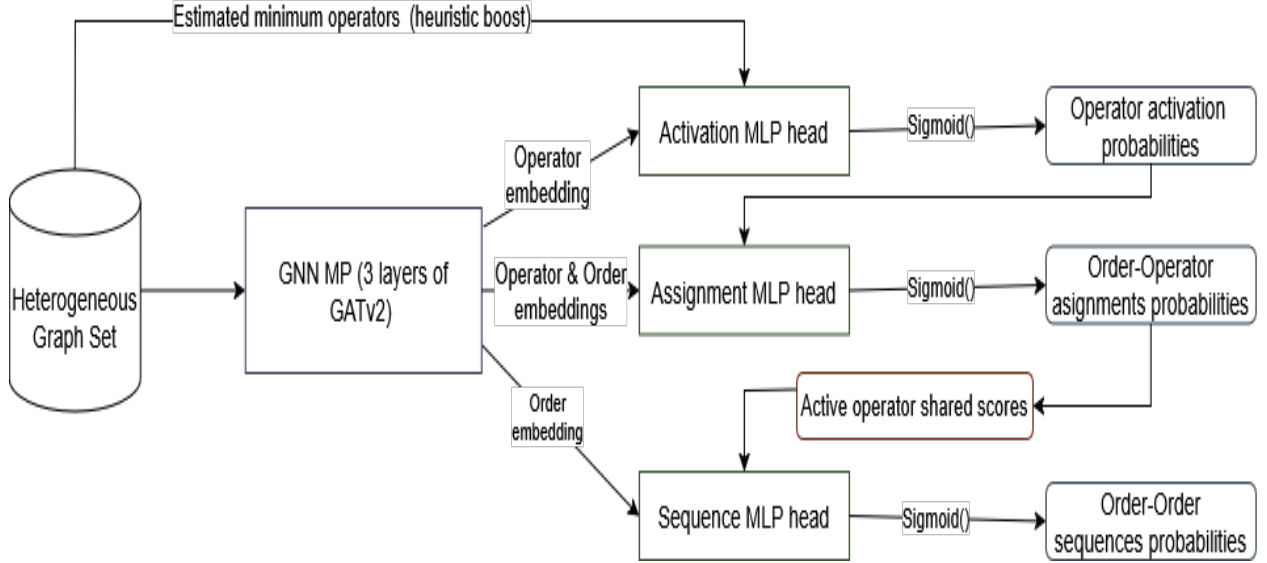


Figure 4.1: One-Shot Architecture of the GNN-MLP Model.

4.2. Recurrent Graph2Seq Model

To overcome the limitations of single-pass prediction, we propose an autoregressive architecture: the **recurrent Graph2Seq model** [XWW+18]. Unlike the one-shot approach, this model separates the static structural understanding of the warehouse from the dynamic, temporal evolution of the schedule. By utilizing a recurrent state mechanism trained via **teacher forcing** (fig. 4.2), the network is explicitly taught to navigate the optimization landscape step-by-step. During training, it learns to imitate the exact sequential chunks of assignments derived from the exact MIP solver. At inference, the network incrementally predicts these concurrent chunks and updates its internal memory to reflect the new state of the environment, ensuring strict adherence to temporal and spatial constraints at every step.

Architecture Overview

As illustrated in fig. 4.3, the model is split into two primary components: a *static encoder* and a *recurrent decoder*.

Static encoder The initial phase processes the complete heterogeneous graph set representing the warehouse batch. A message-passing backbone (three layers of GATv2) analyzes the structural relationships and generates static base embeddings for both ‘operator’ and ‘order’ nodes. These embeddings capture the invariant characteristics of the batch, such as physical order dimensions and baseline operator capabilities.

Recurrent decoder The core of the Graph2Seq model lies in its autoregressive loop, driven by an operator gated recurrent unit (GRU). At each decoding step t , the GRU processes the following dynamic inputs to update the ‘current operators state’:

- The “previous operators state” from step $t - 1$.
- The “last order assignment” per operator (encoding the most recently assigned task).
- The “dynamic” features per operator (e.g., remaining capacity per operator i , current X/Y coordinates)

In addition, a global “fleet context” derived from the aggregated state of all operators is calculated to provide global situational awareness.

These updated, dynamic operator states are then concatenated with the static order embeddings and passed into our three MLP heads:

- **Activation MLP head:** Evaluates whether an operator should be active during the current temporal chunk.
- **Assignment MLP head:** Evaluates the dynamic operator state against the static order embeddings to output a sigmoid probability matrix $\hat{P}_{assign}^{(t)}$ for immediate next assignments.
- **Sequence MLP head:** Utilizes order embeddings and shared active operator scores to predict the immediate sequencing flow $\hat{P}_{seq}^{(t)}$.

Schedule Construction via Step-wise Decoding

Instead of relying on a post-processing greedy algorithm to clean up a static probability matrix, the recurrent model actively participates in the construction of the schedule by looping through sequential prediction steps.

During inference, the decoding loop operates as follows:

1. **State initialization:** The static GNN embeddings are computed, and the GRU hidden states are initialized with operators’ embeddings.
2. **Activation first:** Similarly to the one-shot GNN greedy decoder, the decoder calculates the absolute theoretical minimum of operators required to process the batch (K). The GNN’s activation head ($\hat{P}_{active}$) is used strictly to rank and lock the active fleet to these top K operators.
3. **Step prediction (t):** The MLP heads predict activation, assignment, and sequence probabilities based on the current dynamic state of the warehouse.

4. **Assignment and masking:** The system greedily selects the highest-probability operator-order assignments for the current step. Crucially, any assigned order is immediately marked as completed and mathematically *masked out*, ensuring it cannot be selected by any operator in future steps.
5. **State update ($t \rightarrow t + 1$):** For each operator that received an assignment, their dynamic features are updated [PEL+24]. This includes updating their physical location to the destination of the new order and accumulating the estimated processing and travel times to track capacity against the hard shift horizon (H_{fixed}). These updated features are fed back into the GRU.
6. **Loop termination:** The autoregressive loop continues to select concurrent *non-overlapping* assignments until all orders in the global pool are successfully scheduled, or all active operators have exhausted their temporal capacity. If capacity is exhausted, the model can iteratively activate standby operators from the inactive pool to handle remaining tasks.

This *sequential* approach inherently prevents spatial collisions and duplicate assignments by updating the environment state between decisions. By breaking the massive combinatorial search space into smaller, manageable chunks, the recurrent model can demonstrate significantly improved generalization and strict adherence to constraints compared to the static one-shot architecture.

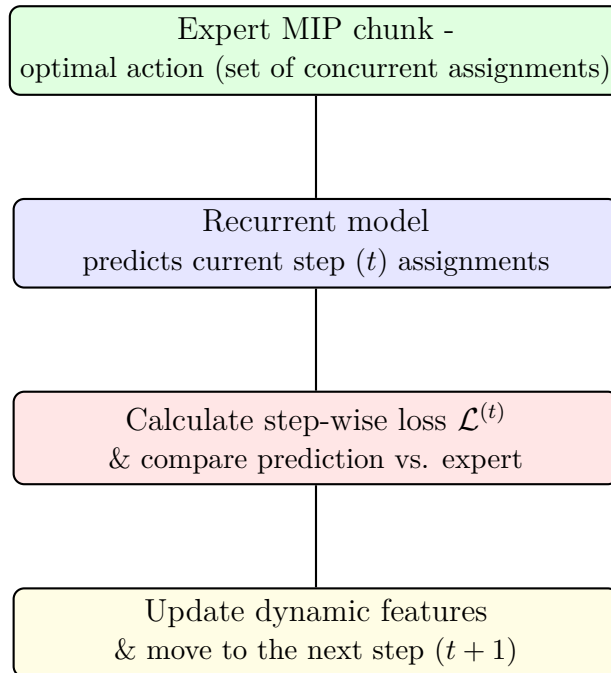


Figure 4.2: Teacher Forcing Pipeline.

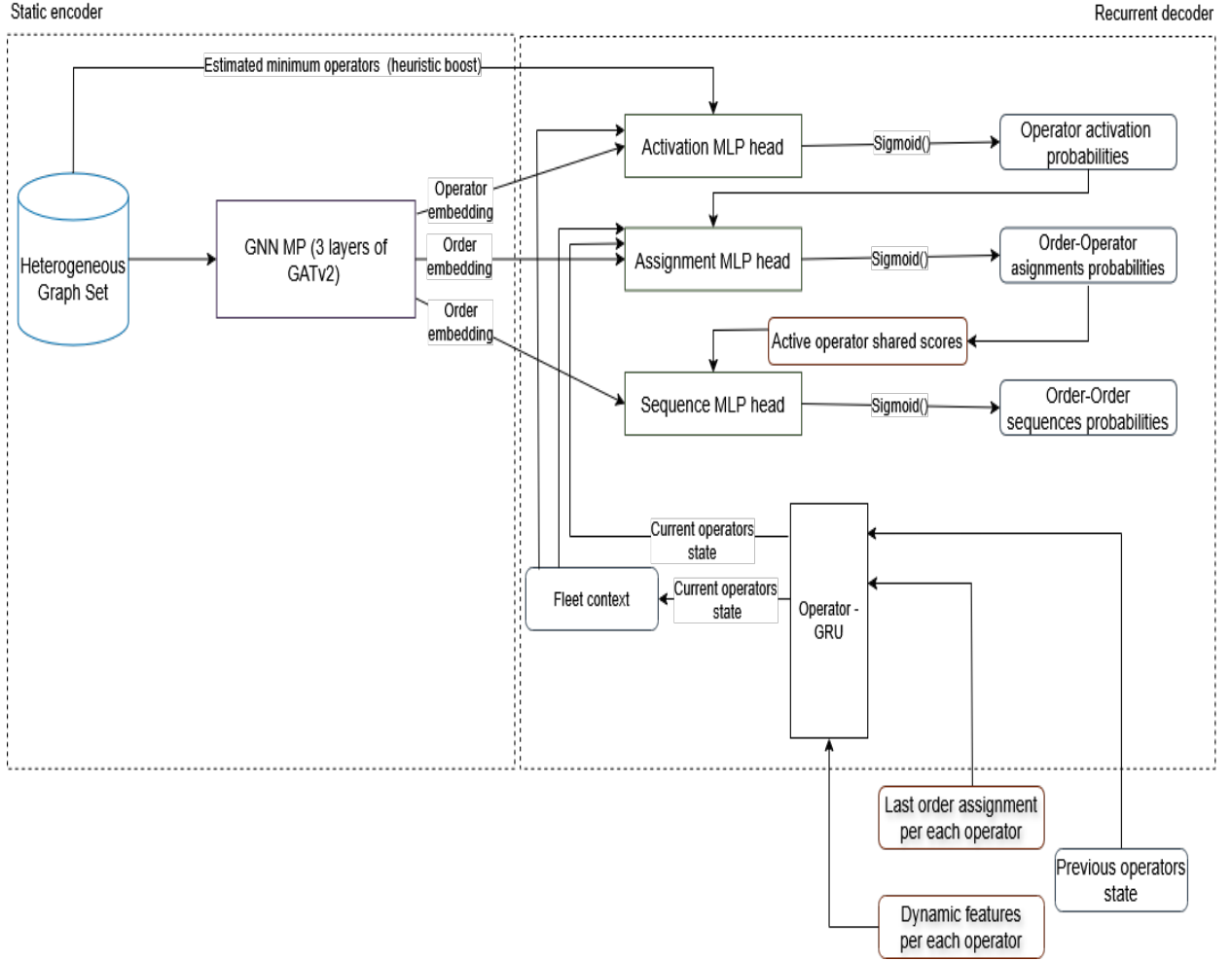


Figure 4.3: Recurrent Architecture of the Graph2Seq Model.

5. Validation Metrics and Key Performance Indicators

To rigorously evaluate the performance of the proposed neural scheduling models, we employ a combination of standard machine learning metrics and domain-specific key performance indicators (KPIs). The evaluation is divided into two continuous phases: training validation and post-training schedule evaluation.

5.1. Training Validation Metrics

During the training phase, the models are evaluated continuously to monitor convergence and prevent overfitting. The two primary metrics utilized are:

- **Loss curves:** The dynamically-weighted binary cross-entropy (BCE) loss is tracked across epochs. For the recurrent model, the step-wise loss is accumulated via back-propagation through time (BPTT). Tracking the training and validation loss curves ensures the models are successfully learning to approximate the exact solver’s policy.
- **F1-score:** Because the scheduling problem is highly sparse (the vast majority of possible order-operator assignments are invalid or sub-optimal), accuracy alone is a misleading metric. We utilize the f1-score to rigorously evaluate the models’ precision and recall in identifying the exact optimal assignments and sequence edges from the massive pool of candidates.

5.2. Domain-Specific KPIs

Once a model generates a complete schedule, its practical utility is measured using five distinct physical and operational KPIs. These metrics reflect real-world warehouse efficiency, labor costs, and fairness:

- **Makespan (Z):** The total global time required to complete the entire batch of orders. Minimizing the makespan is the primary metric for operational speed.
- **Activation score ($\sum y_i$):** The total number of unique operators deployed to complete the batch. Minimizing activation directly translates to reduced labor costs.
- **Weighted combination score:** The primary business objective function defined in eq. 3.1. It calculates the weighted trade-off between the makespan (α) and the activation score (β), representing the **ultimate** operational cost.
- **Total flow time (TFT):** The cumulative time that all orders spend in the system (waiting plus processing time). A lower TFT indicates highly efficient routing and minimal congestion.
- **Coefficient of variation (CV):** A statistical measure of workload fairness across the deployed workforce. It calculates the normalized standard deviation of the operators’

active working times. A lower CV indicates a perfectly balanced schedule where no single operator is unfairly burdened.

5.3. Optimality and Lower Bound Gaps

To translate these raw KPIs into comparative scientific results, each neural model is quantitatively and qualitatively evaluated using **optimality gaps**. For the *mini-batch* datasets, the neural schedules are directly compared against the exact ground truth generated by the MIP solver. The gap represents the percentage deviation from mathematical optimality (e.g., $Gap_{optimality} \leq 5\%$ for excellent imitation, $5\% < Gap_{optimality} \leq 10\%$ for acceptable imitation).

However, for *large-scale* batches where the exact MIP solver cannot execute, we cannot compute a standard optimality gap. Instead, the generalization capabilities of the neural solvers are proven via the **lower bound (LB) gap**. The LB gap measures the models’ performance against the theoretical mathematical limits of the environment (e.g., the absolute minimum time physically required to move all boxes, assuming perfect parallelization and zero routing delay). Evaluating the LB gap ensures that the models’ scalability is analyzed against a rigorous, invariant mathematical standard.

Obtaining a similar LB gaps in both *mini-batch* validation and *full-batch* testing, proves the model generalization of scale-invariant scheduling. Having:

$$\Delta Gap_{LB} = Gap_{LB}^{full} - Gap_{LB}^{mini} \quad (5.1)$$

If ΔGap_{LB} remains small (e.g., $\Delta Gap_{LB} \leq 5\%$ for excellent generalization, $5\% < \Delta Gap_{LB} \leq 10\%$ for acceptable generalization), then the model’s generalization capability is considered mathematically established.

While the lower bound provides an absolute theoretical limit, it often represents an idealized, unachievable state that ignores complex spatial routing constraints. Therefore, for the sake of a realistic comparison in large-scale testing—where exact optimal ground truth is unavailable—a **heuristic greedy baseline** is also utilized. By comparing the neural models against this routing-feasible heuristic, we can effectively quantify the *practical efficiency* gains and operational superiority of the learned neural policy over a standard, rule-based algorithm.

6. Description of Experiments

The experimental framework is designed to comprehensively evaluate the learning capabilities, scheduling optimality, and scalability of the proposed models. The experiments are structured to first validate the models against exact MIP solutions, followed by fine-grained testing of their generalization on real-world, large-scale warehouse environments.

6.1. Dataset Generation and Configurations

To train and evaluate the models, we generate warehouse scheduling instances divided into two distinct categories:

- **Mini-batch instances (training & exact validation):** These consist of small-scale scheduling problems with limited orders and operators. For every instance in this dataset, the exact MIP solver is executed to optimality to extract the ground-truth order-operator assignments and temporal sequences. These exact solutions serve as expert demonstrations for the imitation learning pipeline.
- **Large-scale instances (scalability testing):** These instances reflect realistic, high-volume warehouse operations. Due to the NP-hard nature of the problem, exact MIP optimization is computationally intractable at this scale. These datasets are therefore reserved for testing the zero-shot generalization capabilities of the neural models.

Each training and test example is represented as a **fully connected heterogeneous graph**, where the different node and edge types encode the various entities and relationships involved in the warehouse scheduling problem. This graph-based formulation enables the model to jointly reason over assignment feasibility, structural dependencies, and temporal interactions within a unified relational representation.

To improve robustness and reduce sensitivity to superficial graph indexing, the training set is further enriched through **isomorphic data augmentation**. Specifically, multiple equivalent graph realizations are generated from the same scheduling instance by applying structure-preserving permutations that leave the underlying optimization problem unchanged. This augmentation encourages the model to learn invariant structural patterns rather than overfitting to a single graph ordering.

In addition, a dedicated **hyperparameter tuning** phase is conducted to identify the most effective architectural and optimization settings for each model. The explored hyperparameters include the batch size, the separate learning rates of the shared trunk and each prediction head, the hidden embedding dimension, the number of attention heads, the number of GNN layers, and the dropout rate. This tuning process ensures that the final reported results reflect both a fair and optimized evaluation of the proposed architectures.

6.2. Training Protocol and Architectures

The neural models, specifically leveraging GATv2 edge convolutions for the encoder and specialized MLP heads for assignment and sequencing predictions, are trained using the mini-batch ground truth. The training protocol involves:

- **Learning setup:** The models learn to map the static warehouse graphs to optimal scheduling decisions via supervised learning, minimizing the dynamically-weighted BCE loss.
- **Teacher forcing:** For the recurrent Graph2Seq architecture, training is guided by teacher forcing. The model incrementally processes the state graph (chunk by chunk), with the expert’s exact intermediate decisions fed back into the network to stabilize the learning of dynamic constraints over time.

Beyond the training phase, the architectural designs introduce a significant trade-off regarding computational efficiency during deployment. The one-shot model benefits from an extremely fast forward-pass inference; however, the necessary ad-hoc post-processing required to resolve constraint violations and refine the schedule is heavily time-consuming and scales poorly with respect to the COP dimension. Conversely, the recurrent Graph2Seq model naturally exhibits a larger baseline inference time due to its sequential unrolling over the COP dimension. Nevertheless, this overhead is heavily mitigated by its *non-overlapping chunking* strategy: by predicting multiple concurrent assignments simultaneously at each iteration t , the recurrent model drastically reduces the total number of sequential steps. This results in an efficient inference time that directly guarantees constraint adherence without the need for expensive downstream corrections.

6.3. Evaluation Scenarios

The testing phase is divided into two primary scenarios to rigorously assess the KPIs discussed in the previous section:

- **Small-scale exact validation:** The models are evaluated on unseen mini-batch instances. The neural schedules are directly compared against the MIP ground truth to calculate the **optimality** gaps for makespan, activation score, weighted combination score, TFT and CV. This scenario validates the feature extraction power of the GATv2 layers and the decision-making process of the MLP heads.
- **Large-scale generalization:** To prove *zero-shot* generalization capabilities, the specific model that demonstrates the best overall performance during the mini-batch evaluation is selected and deployed on the large-scale, full-problem instances without any further fine-tuning. Performance is evaluated by comparing the resulting KPIs against the **lower bound** gaps and the greedy heuristic baseline. This scenario demonstrates the practical viability of the autoregressive formulation when handling massive combinatorial spaces where traditional exact solvers fail.

7. Results

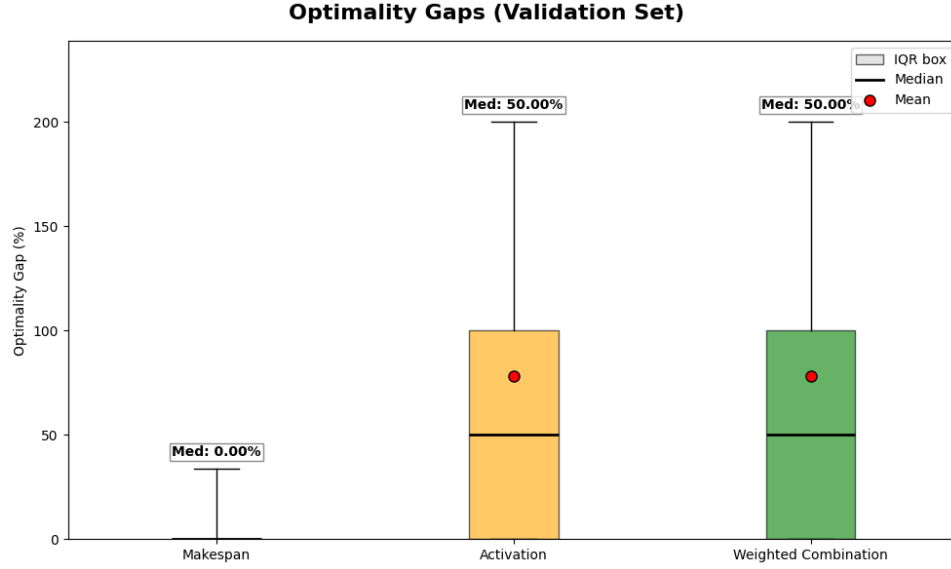
7.1. Comparative Analysis on the Validation Set

To evaluate the quality and robustness of the neural schedules with respect to the MIP optimal solutions, the distribution of the optimality gaps was analyzed over the validation set for the primary objective metrics and the efficiency-fairness-related metrics. Figures 7.1a and 7.1b summarize the gaps for makespan, activation, and the weighted combination score, while figures 7.2a and 7.2b report the corresponding results for TFT and CV. Together, these figures provide a compact view of the median, dispersion, and outlier behavior of the gap distributions, enabling a direct comparison between the one-shot and recurrent architectures.

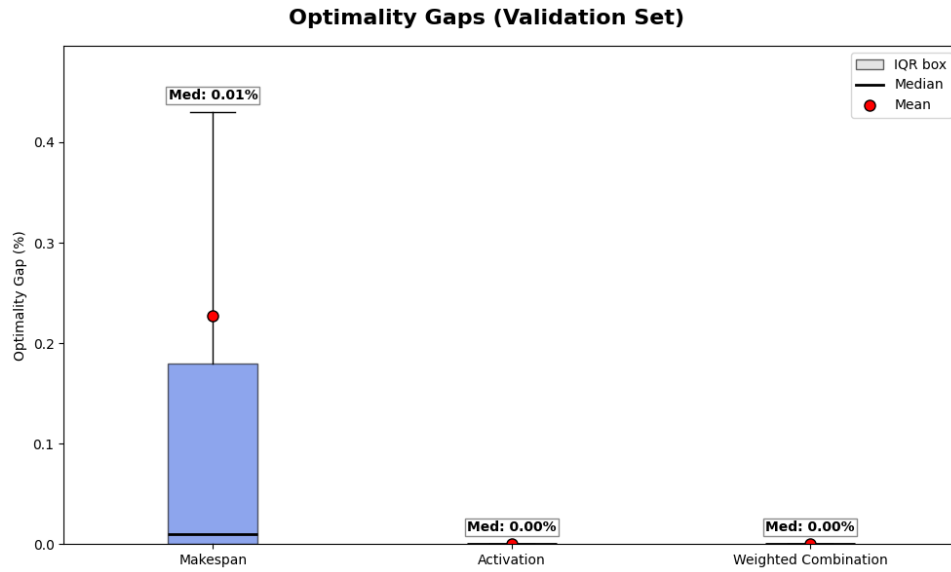
7.1.1. Primary Objective Optimality Gaps

The primary objectives—makespan, activation, and the weighted combination score—reveal the distinct advantages of the sequential approach (see fig. 7.1).

- **One-shot GNN model (fig. 7.1a):** For the one-shot architecture, the optimality gaps are exceptionally high and exhibit severe variance. The median gaps for activation and the weighted combination are both approximately 50.00%, with the means approaching 75%. The makespan gap, while having a median of 0.00%, shows extreme outliers and variance. This indicates that the ad-hoc post-processing required to fix initially infeasible one-shot predictions severely degrades the schedule’s mathematical optimality.
- **Graph2Seq recurrent model (fig. 7.1b):** In contrast, the recurrent model’s optimality gaps are practically zero. The median gaps for makespan, activation, and the weighted combination are all precisely 0.00% (with means of 0.22%, 0.00%, and 0.00% respectively), and the interquartile ranges are incredibly tight. This demonstrates that the recurrent chunking mechanism allows the network to natively learn and output highly optimal, constraint-adhering schedules without requiring specific ad-hoc post-processing.



(a) Primary Objective Optimality Gaps (One-Shot GNN Model)



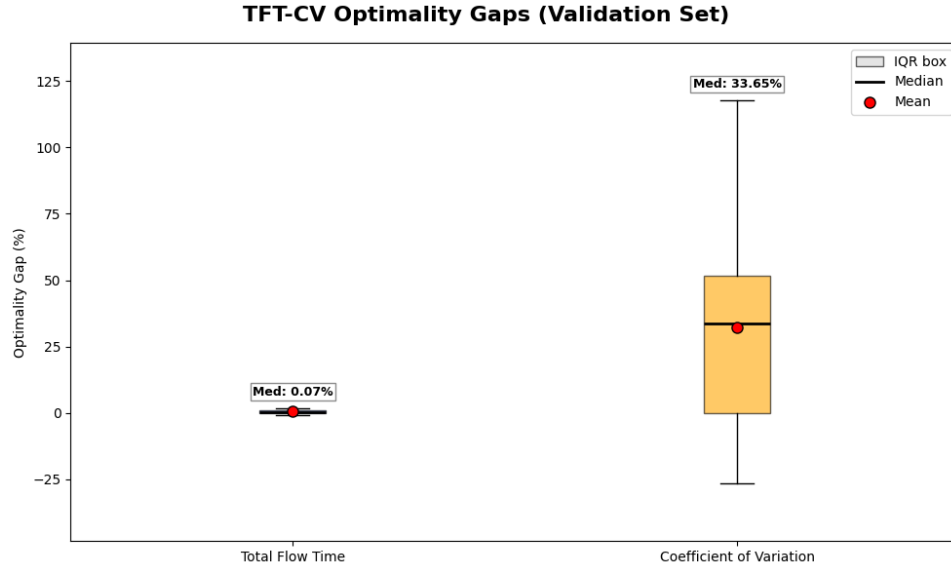
(b) Primary Objective Optimality Gaps (Graph2Seq Recurrent Model)

Figure 7.1: Primary Objective Optimality Gaps Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

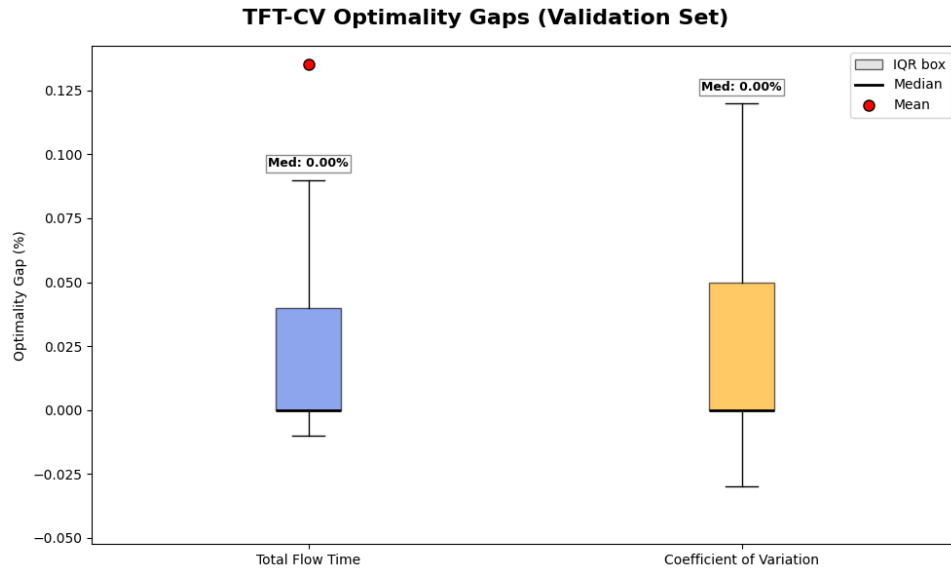
7.1.2. Efficiency and Fairness Optimality Gaps

Similar trends are observed when evaluating routing efficiency, measured via total flow time (TFT), and workforce fairness, measured via the coefficient of variation (CV) (see fig. 7.2).

- **One-shot GNN model (fig. 7.2a):** The one-shot model struggles significantly with fairness (CV), showing a median optimality gap of 33.65% and extreme variance extending beyond 125%. This suggests that the heuristic post-processing phase assigns operators highly uneven workloads. While the TFT median gap is low (0.07%), its distribution still indicates inconsistency in maintaining optimal routing flows.
- **Graph2Seq recurrent model (fig. 7.2b):** The recurrent model again demonstrates superior, near-perfect performance. Both TFT and CV show median gaps of exactly 0.00% with negligible variance. By considering dynamic constraints temporally at each prediction step, the autoregressive model natively balances the workload among the deployed operators while keeping the overall flow times optimal.



(a) TFT & CV Optimality Gaps (One-Shot GNN Model)



(b) TFT & CV Optimality Gaps (Graph2Seq Recurrent Model)

Figure 7.2: Efficiency to Fairness Optimality Gaps Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

7.2. Large-Scale Generalization: Lower Bound and Baseline Analysis

Given the recurrent Graph2Seq model’s evident superiority in aligning with mathematical exactness and efficiently satisfying constraints, it is established as the valid candidate for the large-scale generalization experiments. In these full-problem scenarios, where the combinatorial complexity far exceeds the capabilities of the MIP solver, we evaluate the model’s performance against theoretical LB and a greedy heuristic baseline.

7.2.1. Proof of Generalization: Lower Bound Gap Analysis (Validation vs. Test)

To measure how well the learned policy generalizes to unobserved, massive problem spaces, we analyze the LB gaps. Figure 7.4 visualizes the LB gap distributions for the efficiency and fairness KPIs on the large-scale test set. For the TFT, the median gap relative to the theoretical, unconstrained lower bound is exceptionally tight at -5.56%. This indicates that even in highly complex warehouse layouts, the model routes orders with minimal delay. Similarly, the CV exhibits a median LB gap of 0.02%, proving that the model preserves near-perfect workload fairness, maintaining the same robust performance observed on the small-scale mini-batches.

Furthermore, analyzing the primary objectives (in fig. 7.3) confirms the model’s structural generalization capabilities. Notably, the median makespan gap operates in negative area (-7.65%), while the activation gap remains low (median 5.88%). This signifies that the recurrent model is effectively optimizing the highly coupled spatio-temporal problem, keeping operational costs exceptionally close to the theoretical limits without the aid of expert demonstrations at this scale.

A negative validation-to-test LB gap for makespan or TFT does not indicate that the model’s temporal efficiency scales perfectly beyond the mini-batch limits; rather, it reflects a strategic trade-off where the model compensates for the full-problem complexity by activating more operators, yielding a positive activation gap.

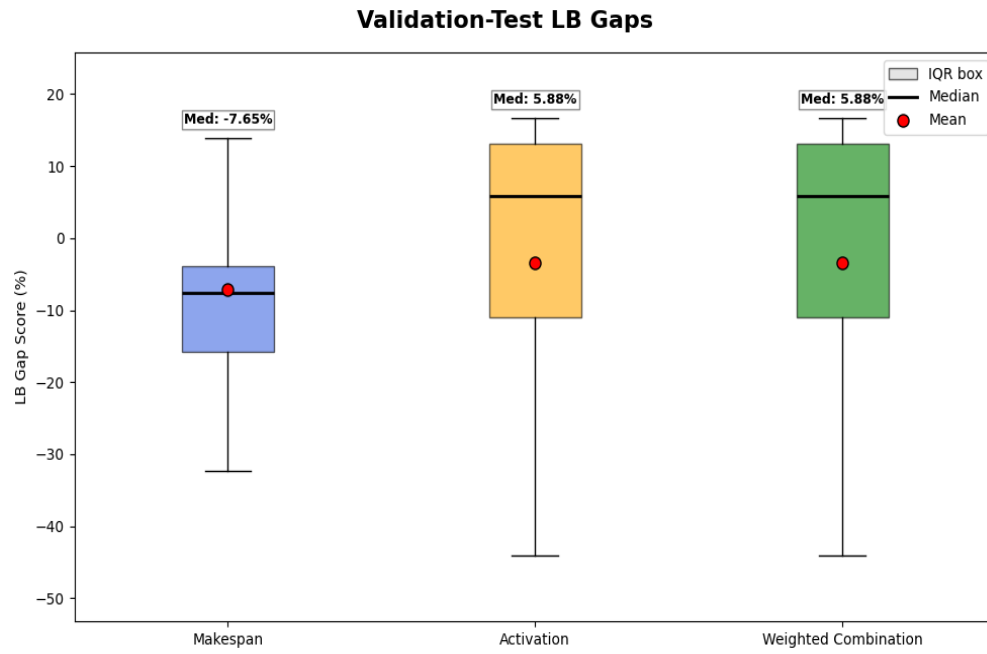


Figure 7.3: Validation vs. Test Lower Bound Gaps (Primary Objective KPIs)

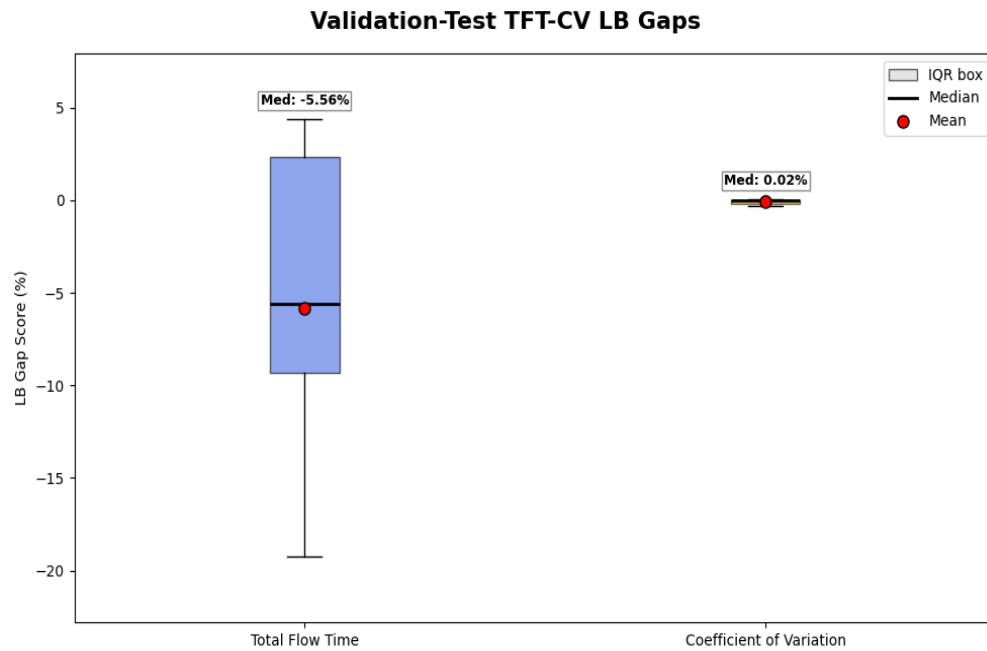


Figure 7.4: Validation vs. Test Lower Bound Gaps (Efficiency and Fairness KPIs)

7.2.2. Comparison Against the Heuristic Greedy Baseline

While lower bounds provide an absolute theoretical limit, comparing the neural model to a functional dispatching algorithm highlights its practical superiority. Figure 7.5 presents the comparison of the ultimate business objective—the weighted combination score—between the theoretical LB, the recurrent neural schedule, and the greedy heuristic algorithm.

The plot reveals a clear contrast in performance scaling. As the combinatorial complexity increases, the greedy heuristic’s performance deteriorates significantly, diverging far from the optimal frontier due to its inability to forecast complex routing interactions regarding the combinatorial nature. Conversely, the recurrent Graph2Seq model maintains a tight, stable trajectory close to the theoretical LB. This stable gap confirms that the autoregressive neural policy successfully keeps a holistic understanding of the warehouse environment, vastly outperforming rule-based heuristics and validating its applicability for large-scale, real-world deployment.

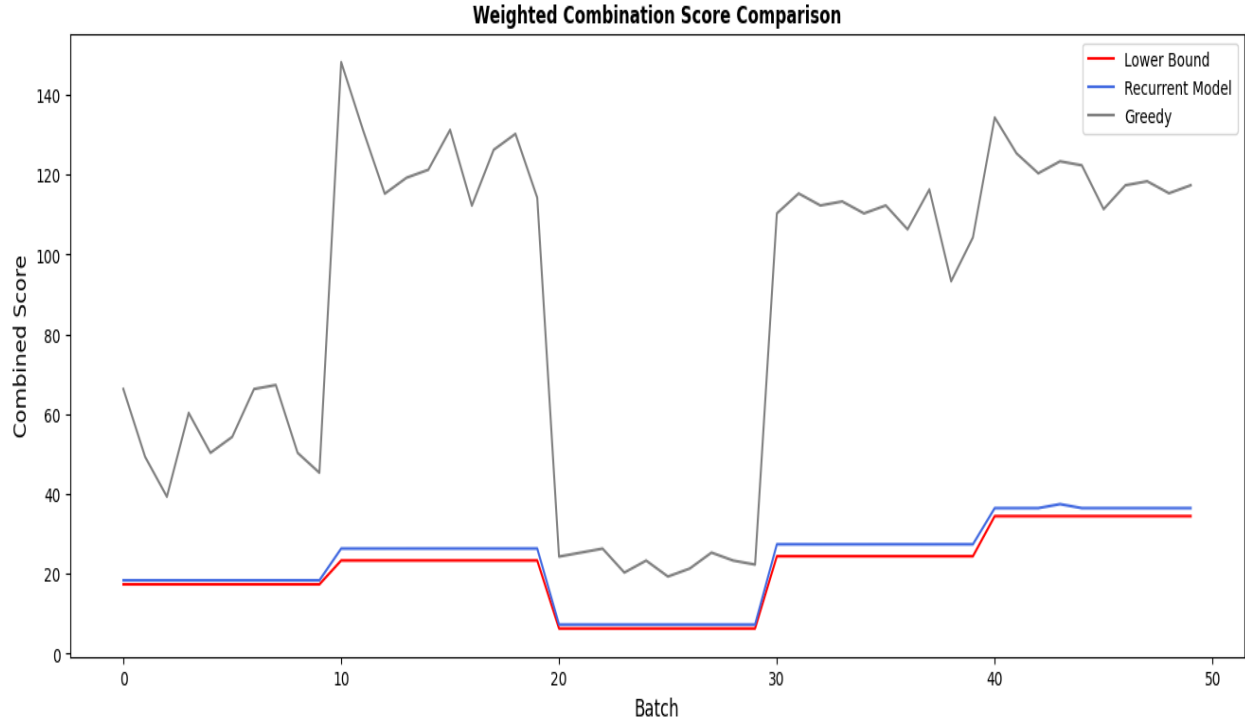


Figure 7.5: Weighted Combination Score Comparison: Lower Bound, Recurrent Model, and Heuristic Greedy Algorithm

8. Computational Environment and Used Technologies

The experiments were implemented in python using **Pandas** for data manipulation, **NumPy** for numerical operations, **PyTorch** together with **PyTorch Geometric** for graph-based learning, **Matplotlib** for data visualization. The exact optimization problems were solved with a MIP-based framework (**Pyomo** for modelling, **IBM ILog CPLEX** for solving), and the resulting schedules were used as supervision for the neural model. The software stack also included libraries for **Scikit-learn** for essential preprocessing and clustering, whereas **Matplotlib** for data visualization.

All experiments were executed in a GPU-capable environment to accelerate both training and inference. This was necessary because the workflow combined exact optimization, graph neural network training, and repeated evaluation across many generated mini-batches. The computational environment was therefore selected to ensure both reproducibility and practical runtime efficiency.

The implementation relied on a modular pipeline that separated data preparation, graph construction, solver-based label generation, and neural training. This structure made it possible to reuse the same preprocessing logic across the different experimental settings, including the mini-batch validation analysis and the full-scale test evaluation. In addition, the environment supported repeated runs with fixed seeds and consistent data transformations, helping to reduce variance across experiments and improve the reliability of the reported results.

For the exact optimization stage, the solver was used offline to generate reference schedules rather than during deployment. This allowed the neural model to learn from high-quality supervision while keeping the training procedure computationally manageable. Once trained, the recurrent model could be evaluated independently of the exact solver, which is essential for studying scalability on large instances where exact optimization becomes impractical.

Overall, the computational environment was chosen to balance three requirements: numerical stability, reproducibility, and scalability. The combination of GPU acceleration, graph-based deep learning tools, and exact offline supervision provided a practical foundation for comparing neural and heuristic scheduling strategies under realistic warehouse constraints.

References

- [AAS25] D. Angioni, C. Archetti, and M. G. Spera. Neural combinatorial optimization: a tutorial. *Science Direct*, 2025.
- [And25] A. M. Andoni I. Garmendia Josu Ceberio. Neural combinatorial optimization: a new player in the field [about generalization to different instance sizes]. *Arxiv - Cornell University*, 2025.
- [PEL+24] D. Pugacheva, A. Ermakov, I. Lyskov, I. Makarov, and Y. Zotov. Enhancing GNNs performance on combinatorial optimization by recurrent feature update. *arXiv - Cornell University*, 2024.
- [Pin08] M. L. Pinedo. *Scheduling Theory, Algorithms, and Systems*. 3rd ed. Springer, 2008.
- [Wol21] L. Wolsey. *Integer Programming*. Wiley Interscience, 2021.
- [XWW+18] K. Xu, L. Wu, Z. Wang, Y. Feng, M. Witbrock, and V. Sheinin. Graph2Seq: graph to sequence learning with attention-based neural networks. *arXiv - Cornell University*, 2018.
- [ZLZ24] Y. Zhao, X. Luo, and Y. Zhang. The application of heterogeneous graph neural network and deep reinforcement learning in hybrid flow shop scheduling problem. *Science Direct*, 2024.